

AtriumX

Complete Build Specification

Campus Marketplace Platform — Full Rebuild Document

This document specifies every workflow, data structure, color token, plan rule, Supabase table, RLS policy, and SQL function required to build AtriumX from scratch. No hardcoded data. No placeholder content. All data must be live from Supabase. Where a screen has no data, it shows an empty state message — never fabricated content.

1. Design System

COLOR TOKENS — USE THESE CLASS NAMES ONLY, NEVER RAW HEX IN JSX

Token	Hex Value	Usage
bg-slate-deep	#0D2538	Page background — every screen
bg-slate-card	#0F2D45	Card and input backgrounds
border-slate-border	#1E3A4F	All borders and dividers
bg-teal-primary / text-teal-primary	#1A5F7A	Primary brand colour, active states, buttons
teal-light	#2A7A9A	Hover states on teal elements
teal-faint	#1A5F7A22	Subtle teal backgrounds, highlights
text-cream	#FFF7ED	Primary text on dark backgrounds
text-cream-muted	#C9B99A	Secondary and placeholder text
text-gold / bg-gold	#FBBF24	Prices, plan badges, accents
gold-muted	#D97706	Gold hover states
bg-ember / text-ember	#F97316	Primary CTA buttons, brand accent
ember-dark	#C2550F	Ember hover state
text-red-400	#F87171	Error messages, destructive actions
text-amber-400	#FBBF24	Warnings, pending status

TYPOGRAPHY

Font family sans: DM Sans (Google Fonts). Font family serif: DM Serif Display (Google Fonts). Import both in index.html or index.css. Never substitute with system fonts.

Heading sizes: page titles use font-serif text-3xl. Section labels use font-bold text-base. Body copy uses text-sm. Helper text uses text-xs.

AVATAR COLORS — EXACTLY THESE SIX, ASSIGNED RANDOMLY ON REGISTRATION

#0F6E56 #185FA5 #993C1D #993556 #534AB7 #3B6D11

INPUT CLASS — REUSE THIS STRING ON EVERY INPUT, SELECT, AND TEXTAREA

"w-full bg-slate-card border border-slate-border rounded-xl px-4 py-3
text-cream text-sm placeholder:text-cream-muted
focus:outline-none focus:border-teal-light transition-colors"

2. Plan Tiers

■ All plan restrictions must be enforced in code, not just displayed. Every field in this table is a hard constraint.

Property	Ghost	Flash	Visible	Loud	Unmissable
Price	Free	R9	R29	R79	R149
Days active	3	2	7	14	30
maxListings	1	1	3	8	999
maxPhotos	0	1	1	3	5
maxVariants	0	0	3	8	999
maxMsgs/conv	3	10	999	999	999

canRenew	X	X	✓	✓	✓
canNegBadge	X	✓	✓	✓	✓
verified badge	X	X	X	✓	✓
analytics	X	X	X	✓	✓
video upload	X	X	X	X	✓
spotlight	X	X	X	X	✓

PLAN ENFORCEMENT RULES

1. Before rendering PostListing form, count listings with status active OR pending for this seller. If count >= PLAN_TIERS[plan].maxListings, show limit wall — do not render the form.
2. Photo upload slots: render exactly PLAN_TIERS[plan].maxPhotos upload buttons. If maxPhotos is 0, hide the photo section entirely and show a text-only notice.
3. Variant section: only render if PLAN_TIERS[plan].maxVariants > 0. Cap at maxVariants entries.
4. Message input: after seller sends PLAN_TIERS[plan].maxMsgs messages in one conversation, disable the input and show upgrade prompt. Buyer is never limited.
5. Renew button on ListingDetail: only render if PLAN_TIERS[plan].canRenew is true AND listing.status is not suspended.
6. Negotiation badge: only render "Open to offers" chip if PLAN_TIERS[plan].canNegBadge is true AND listing.is_negotiable is true.

3. File Structure

■ **Consolidate logic. If two components share a parent page and have no independent reuse case, put them in the same file. Fewer files = fewer import errors.**

REQUIRED FILES — DO NOT CREATE ADDITIONAL FILES BEYOND THIS LIST

src/services/supabaseClient.ts	Supabase client initialisation only. No logic.
src/services/dataService.ts	ALL database calls. Every function that touches Supabase goes here. No component may call supabase directly except for realtime subscriptions in ChatWindow.
src/context/AppContext.tsx	Global state: currentUser, toasts, authPromptOpen, redirectAfterLogin, unreadMessageCount, isLoadingAuth. No other state lives here.
src/pages/Entrance.tsx	Welcome screen. Logo + AtriumX name + Student / Business cards. No auth check.
src/pages/StudentAuth.tsx	Login and Register in one file. Toggle between modes via ?mode=register param.
src/pages/Feed.tsx	Listing feed with search bar and category chips. Calls getListings(). No mock data ever.
src/pages/ListingDetail.tsx	Full listing view. Photo gallery with thumbnails. Video player if video_url exists. Interest button, Mark Sold, Renew, Report. ReportModal rendered inline at bottom of this file — no separate file needed.
src/pages/PlanSelect.tsx	Plan selection cards. If user has active non-ghost plan, show current plan + expiry + continue button at top. Clicking a card navigates immediately to /post with { state: { plan: key } }.
src/pages/PostListing.tsx	Listing form. Reads plan from location.state. Adapts all fields to plan tier. Poster mode toggle. Limit wall if at maxListings. Submission confirmation screen after success.
src/pages/Profile.tsx	Public profile view. Shows active listings grid, sold toggle, pending count notice if own profile, Edit Profile button if own profile, Logout button if own profile.
src/pages/EditProfile.tsx	Edit name, residence, avatar colour. 6 colour swatches with live preview. Calls updateProfile() then navigates back.
src/pages/ChatPage.tsx	Conversation list on left, ChatWindow on right (desktop) or full screen (mobile). ChatWindow component defined in same file — do not separate it.
src/pages/AdminPanel.tsx	Admin only. Three tabs: Pending, All Listings, Flagged. Approve/Reject/Suspend/Clear actions.
src/pages/RetailerLanding.tsx	Business marketing page. Static content. Links to /retailer/signup.
src/pages/RetailerSignup.tsx	Business application form. Calls submitBusinessApplication(). Success screen after submit.
src/components/common/Navbar.tsx	Top bar: logo.png (h-12) links to /feed. Right side: NotificationBell + avatar circle (links to /profile/:id) when logged in. Sign In button when logged out.
src/components/common/BottomNav.tsx	Bottom tabs: Feed, Post, Messages (logged in only), My Listings (logged in only). Unread badge on Messages.
src/components/common/AuthPromptModal.tsx	Login prompt overlay. Shown when guest attempts protected action.
src/components/common/Toast.tsx	Toast notification component. Rendered from AppContext toasts array in App.tsx.
src/components/common/NotificationBell.tsx	Bell icon with red dot. Fetches getUnreadNotifications() on click. Dropdown list. Marks read on tap.
src/App.tsx	Router, providers, ToastLayer, ModalLayer. All routes defined here.
public/logo.png	Logo file. Must be in public/ folder for Vite to serve it at /logo.png.

4. Workflows

4.1 Registration

- User lands on Entrance. Clicks Enter the Marketplace. Navigates to /feed.
- If user attempts protected action (Post, Messages, My Listings), AuthPromptModal appears.
- User clicks Sign In → navigates to /student?mode=login or /student?mode=register.
- Registration fields: full_name (required), email (required, valid format), password (min 8 chars), residence (required free text).
- On submit: supabase.auth.signUp() with metadata {full_name, residence, avatar_initials, avatar_color, email}.
- Retry loop polls getUserById() up to 5 times at 600ms intervals waiting for trigger to create profile row.
- If trigger fails after 5 attempts, insert profile row manually as fallback.
- On success: setCurrentUser(profile), navigate to redirectAfterLogin or /feed.
- No email confirmation. No waiting period. Instant access.
- No password reset flow exists in this version.

4.2 Login

- Fields: email, password. Calls loginWithEmail().
- On wrong credentials: show "Incorrect email or password. Please try again."
- On success: setCurrentUser(profile), navigate to redirectAfterLogin or /feed.
- Session restored on every page load via restoreSession() in AppContext useEffect.
- Logout available from Profile page. Calls logout(), setCurrentUser(null), navigate to /.

4.3 Posting a Listing

- BottomNav Post tab → if not logged in, show AuthPromptModal.
- If logged in, navigate to /plan-select.
- PlanSelect: if user has active non-ghost plan (plan !== ghost AND plan_expires_at > now), show current plan banner with Continue button at top.
- Clicking a plan card immediately navigates to /post with { state: { plan: key } }. No separate confirm button.
- PostListing: reads plan from location.state. If no plan in state, redirect to /plan-select.
- Check listing limit: count active+pending listings for this seller. If >= maxListings, show wall screen with Upgrade button. Do not render form.
- Form adapts to plan: photos only if maxPhotos > 0, variants only if maxVariants > 0, negotiation checkbox only if canNegBadge.
- Poster mode toggle available on plans with maxPhotos > 0. In poster mode: hide all text fields, show single image upload. On submit: title="Poster listing", description="", price=0, category="other".
- On submit: createListing() sets status=pending. Show confirmation screen with Back to Feed button.
- Listing does not appear on feed until admin approves it.

4.4 Browsing and Discovery

- Feed shows all listings with status=active ordered by created_at descending.
- Category chips filter by category. Search bar filters by title (case-insensitive ilike).
- Listing card shows: first image or placeholder, category chip, title, price (or "From R X" if variants exist), seller avatar+name, interest count, countdown timer.
- Countdown: >= 2 days shows "Xd left" in cream-muted. < 48 hours shows "Xh left" in yellow-400. Expired shows "Expired" in red-400. Recalculates every 60 seconds.
- Tapping card navigates to /listing/:id.
- Feed never shows pending, expired, sold, or suspended listings.

- If feed returns 0 listings, show empty state: "No listings yet. Be the first to post." with Post button.
- No mock data. If database is empty, show empty state.

4.5 Listing Detail

- Fetch listing by id with seller profile joined.
- If listing not found, show "Listing not found" with Back button.
- Increment view_count on every load.
- If video_url exists: show video player at top with first image as poster frame.
- If multiple image_urls: show main image with thumbnail strip below. Tapping thumbnail swaps main image.
- Show: category, listing_type, negotiation badge (if is_negotiable AND canNegBadge), title, price or From price, variants table if exists, expiry countdown, interest count, residence, description, seller card.
- Seller card taps to /profile/:sellerId.
- Buyer sees: "I'm Interested — Message Seller" button. Calls startConversation(). On success navigates to /chat/:convId.
- If listing.status !== active: button disabled, shows "Listing Unavailable".
- Seller sees: Mark as Sold button + Renew button (if canRenew AND status !== suspended).
- Report button: visible to logged-in non-sellers. Opens ReportModal (defined in same file). Modal is centred overlay, not bottom sheet.
- ReportModal: explains report process, single Confirm Report button, calls reportListing(), shows toast, closes.

4.6 Messaging

- startConversation() checks for existing conversation by listing_id + buyer_id first. Returns existing if found — never duplicates.
- ChatPage: left panel shows all conversations for currentUser. Each row shows other party avatar, name, listing title, Done badge if resolved.
- On desktop: list left, ChatWindow right. On mobile: list full screen, tapping a row shows ChatWindow full screen with back arrow.
- ChatWindow: fetch messages on mount. Subscribe to realtime INSERT on messages table filtered by conversation_id.
- Messages: own messages right-aligned in ember, received left-aligned in slate-card.
- Phone masking: replace /d{7,}/g with "[number hidden]" at render time only. Raw content stored in DB.
- Message limit: after seller sends maxMsgs messages in this conversation, disable input, show upgrade prompt. Buyer never limited.
- Resolve button: seller only, conversation not yet resolved. On click: markConversationResolved(), show RatingModal.
- RatingModal: 1-5 star picker, optional comment max 150 chars. Submit calls submitRating(). After submit or skip: call onResolved() which updates conversation state to is_resolved=true.
- Resolved conversations: input replaced with "This conversation is resolved." text.
- New messages arrive via realtime without page refresh. After new message arrives, recalculate unread count via getUnreadMessageCount() and update AppContext.

4.7 Ratings

- RatingModal triggered after seller resolves a conversation.
- Rating has star score 1-5 (required) and optional comment max 150 chars.
- submitRating(sellerId, buyerId, listingId, stars, comment) inserts into ratings table.
- Supabase trigger on_rating_inserted recalculates seller avg_rating and total_ratings on profiles table automatically.
- Skip button closes modal and calls onResolved() with no DB insert.
- Ratings are permanent. No edit or delete.

4.8 Notifications

- Bell icon in Navbar top-right. Red dot when unread notifications exist.
- On bell click: fetch getUnreadNotifications(userId). Show dropdown list.

- If no notifications: show "No new notifications yet. You will be notified when your listing is approved or someone rates you."
- Notification types: `listing_approved` (navigate to `/listing/:listing_id`), `listing_rejected` (navigate to `/listing/:listing_id`), `rating_request` (navigate to `/chat`).
- Each row: icon by type, message text, relative time.
- Tapping a row: `markNotificationRead(id)`, navigate to relevant page, close dropdown.
- Clicking outside dropdown: close without marking read.
- Realtime subscription on notifications table for `currentUser.id`.
- When new notification arrives via realtime: add to list, show red dot.
- `getUnreadNotifications()` is called on bell click only — not on mount.

4.9 Profile

- Public profile at `/profile/:userId`. Fetch profile + listings for that `userId`.
- Shows: avatar circle, `full_name`, residence, `avg_rating` + `total_ratings` (hidden if 0 ratings), `total_listings`, joined date.
- Active listings grid. If 0 active but has pending: show "Your listings are awaiting approval and will appear here once approved."
- Sold listings: collapsed under toggle showing count.
- If own profile: show pending count notice above edit button, Edit Profile button, Logout button.
- Logout: calls `logout()`, `setCurrentUser(null)`, navigate to `/`.
- Edit Profile: pre-fills `full_name`, residence, `avatar_color`. 6 colour swatches with live avatar preview. Calls `updateProfile()`. Navigates back to `/profile/:id` on save.
- Residence field in `EditProfile`: shows autocomplete suggestions from `getResidences()` via `datalist`. Case-insensitive match. Still allows free text.

4.10 Admin Panel

- Route `/admin`. Protected: if `!currentUser?.is_admin` redirect to `/feed` silently.
- `isLoadingAuth` must be in `useEffect` dependency array to prevent infinite loading.
- Three tabs: Pending Approval, All Listings, Flagged.
- Pending: `getPendingListings()`. Each card shows title (or image if poster listing), seller name, category, residence, timestamp. Approve calls `approveListingById()` which sets `status=active` AND inserts `listing_approved` notification. Reject calls `rejectListingById()` which sets `status=suspended` AND inserts `listing_rejected` notification.
- All Listings: `getAllListingsAdmin()`. Filterable by status chip. Suspend button on active listings calls `rejectListingById()`. No notification on suspend.
- Flagged: listings where `report_count > 0`. Clear Reports button calls `clearReports()` which deletes from reports table and resets `report_count` to 0.
- No Navbar — use only `BottomNav`. Sticky header shows "Admin Panel" text.
- Grant admin access: `UPDATE public.profiles SET is_admin = true WHERE email = your-email;`

4.11 Business Portal

- `RetailerLanding` at `/retailer`: marketing page. Business types, packages, how it works. Link to `/retailer/signup`.
- `RetailerSignup` at `/retailer/signup`: fields: `business_name`, `business_type` (dropdown), `custom_business_type` (if Other selected), `contact_person`, email, phone, `selected_package` (dropdown), description (max 300 chars).
- All fields required. On submit: `submitBusinessApplication()`. Show success screen with applicant email.
- No email is sent. No login created. Application stored in `business_profiles` table for manual review.
- Uses Navbar (not `BottomNav`) — retailer portal is separate from student app.

5. Supabase — Complete SQL

■ Run the nuclear reset SQL first to wipe all existing tables and storage. Then run the full schema below. Do not run partial sections.

STEP 1 — RESET (ALREADY PROVIDED BY DEVELOPER, RUN FIRST)

STEP 2 — FULL SCHEMA (RUN THIS AFTER RESET)

Extensions

```
-- Required extensions
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

Profiles Table

```
CREATE TABLE public.profiles (
  id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email text NOT NULL,
  full_name text NOT NULL DEFAULT '',
  residence text,
  avatar_initials text NOT NULL DEFAULT '',
  avatar_color text NOT NULL DEFAULT '#0F6E56',
  plan text NOT NULL DEFAULT 'ghost'
  CHECK (plan IN ('ghost', 'flash', 'visible', 'loud', 'unmissable')),
  plan_expires_at timestamptz,
  avg_rating numeric(3,1) NOT NULL DEFAULT 0,
  total_ratings int NOT NULL DEFAULT 0,
  total_listings int NOT NULL DEFAULT 0,
  is_verified boolean NOT NULL DEFAULT false,
  is_admin boolean NOT NULL DEFAULT false,
  watched_residences text[] NOT NULL DEFAULT '{}',
  joined_date timestamptz NOT NULL DEFAULT now(),
  created_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
```

Residences Table

```
CREATE TABLE public.residences (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text UNIQUE NOT NULL,
  created_at timestamptz DEFAULT now()
);
ALTER TABLE public.residences ENABLE ROW LEVEL SECURITY;
```

Listings Table


```

CREATE TABLE public.listings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  seller_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  title text NOT NULL,
  description text NOT NULL DEFAULT '',
  price numeric(10,2) NOT NULL DEFAULT 0,
  category text NOT NULL,
  custom_category text,
  image_urls text[] NOT NULL DEFAULT '{}',
  video_url text,
  residence text NOT NULL,
  listing_type text NOT NULL DEFAULT 'single'
  CHECK (listing_type IN ('single','ongoing')),
  is_negotiable boolean NOT NULL DEFAULT false,
  plan_tier text NOT NULL DEFAULT 'ghost',
  status text NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending','active','sold','expired','suspended')),
  report_count int NOT NULL DEFAULT 0,
  contact_count int NOT NULL DEFAULT 0,
  view_count int NOT NULL DEFAULT 0,
  variants jsonb NOT NULL DEFAULT '[]',
  expires_at timestampz NOT NULL,
  created_at timestampz NOT NULL DEFAULT now()
);
ALTER TABLE public.listings ENABLE ROW LEVEL SECURITY;

```

Conversations Table

```

CREATE TABLE public.conversations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id uuid NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,
  buyer_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  seller_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  is_resolved boolean NOT NULL DEFAULT false,
  created_at timestampz NOT NULL DEFAULT now(),
  UNIQUE(listing_id, buyer_id)
);
ALTER TABLE public.conversations ENABLE ROW LEVEL SECURITY;

```

Messages Table

```

CREATE TABLE public.messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id uuid NOT NULL REFERENCES public.conversations(id) ON DELETE CASCADE,
  sender_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  content text NOT NULL,
  read boolean NOT NULL DEFAULT false,
  sent_at timestampz NOT NULL DEFAULT now()
);
ALTER TABLE public.messages ENABLE ROW LEVEL SECURITY;

```

Ratings Table

```

CREATE TABLE public.ratings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  seller_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  buyer_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  listing_id uuid NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,
  stars int NOT NULL CHECK (stars BETWEEN 1 AND 5),
  comment text,
  created_at timestampz NOT NULL DEFAULT now()
);
ALTER TABLE public.ratings ENABLE ROW LEVEL SECURITY;

```

Reports Table

```
CREATE TABLE public.reports (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  listing_id uuid NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,  
  reporter_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,  
  reason text,  
  created_at timestamptz NOT NULL DEFAULT now()  
);  
ALTER TABLE public.reports ENABLE ROW LEVEL SECURITY;
```

Notifications Table

```
CREATE TABLE public.notifications (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,  
  type text NOT NULL,  
  message text NOT NULL,  
  listing_id uuid REFERENCES public.listings(id) ON DELETE SET NULL,  
  conversation_id uuid REFERENCES public.conversations(id) ON DELETE SET NULL,  
  read boolean NOT NULL DEFAULT false,  
  created_at timestamptz NOT NULL DEFAULT now()  
);  
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
```

Business Profiles Table

```
CREATE TABLE public.business_profiles (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  business_name text NOT NULL,  
  business_type text NOT NULL,  
  custom_business_type text,  
  contact_person text NOT NULL,  
  email text NOT NULL,  
  phone text NOT NULL,  
  selected_package text NOT NULL,  
  description text,  
  created_at timestamptz NOT NULL DEFAULT now()  
);  
ALTER TABLE public.business_profiles ENABLE ROW LEVEL SECURITY;
```

Push Preferences Table

```
CREATE TABLE public.push_preferences (  
  user_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,  
  push_enabled boolean NOT NULL DEFAULT false,  
  updated_at timestamptz DEFAULT now()  
);  
ALTER TABLE public.push_preferences ENABLE ROW LEVEL SECURITY;
```

RLS Policies

Profiles RLS

```
-- Anyone can read profiles
CREATE POLICY "profiles_select_all" ON public.profiles
FOR SELECT USING (true);

-- Users can update their own profile
CREATE POLICY "profiles_update_own" ON public.profiles
FOR UPDATE TO authenticated
USING (id = auth.uid()) WITH CHECK (id = auth.uid());

-- Trigger inserts profile – no INSERT policy needed for users
-- Admins can update any profile (for is_admin flag)
CREATE POLICY "profiles_insert_trigger" ON public.profiles
FOR INSERT WITH CHECK (true);
```

Residences RLS

```
CREATE POLICY "residences_select_all" ON public.residences
FOR SELECT USING (true);

CREATE POLICY "residences_insert_auth" ON public.residences
FOR INSERT TO authenticated WITH CHECK (true);
```

Listings RLS

```
CREATE POLICY "listings_select_active" ON public.listings
FOR SELECT USING (status = 'active' OR seller_id = auth.uid());

CREATE POLICY "listings_insert_own" ON public.listings
FOR INSERT TO authenticated WITH CHECK (seller_id = auth.uid());

CREATE POLICY "listings_update_own" ON public.listings
FOR UPDATE TO authenticated USING (seller_id = auth.uid());

-- Admin select all (for admin panel)
CREATE POLICY "listings_select_admin" ON public.listings
FOR SELECT USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND is_admin = true)
);

-- Admin update any listing (approve/reject)
CREATE POLICY "listings_update_admin" ON public.listings
FOR UPDATE USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND is_admin = true)
);
```

Conversations RLS

```
CREATE POLICY "conversations_insert_buyer" ON public.conversations
FOR INSERT TO authenticated WITH CHECK (buyer_id = auth.uid());

CREATE POLICY "conversations_select_participant" ON public.conversations
FOR SELECT TO authenticated
USING (buyer_id = auth.uid() OR seller_id = auth.uid());

CREATE POLICY "conversations_update_participant" ON public.conversations
FOR UPDATE TO authenticated
USING (buyer_id = auth.uid() OR seller_id = auth.uid());

CREATE POLICY "conversations_delete_participant" ON public.conversations
FOR DELETE TO authenticated
USING (buyer_id = auth.uid() OR seller_id = auth.uid());
```

Messages RLS

```
CREATE POLICY "messages_insert_own" ON public.messages
FOR INSERT TO authenticated
WITH CHECK (
  sender_id = auth.uid() AND
  EXISTS (
    SELECT 1 FROM public.conversations
    WHERE id = conversation_id
    AND (buyer_id = auth.uid() OR seller_id = auth.uid())
  )
);

CREATE POLICY "messages_select_participant" ON public.messages
FOR SELECT TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.conversations
    WHERE id = conversation_id
    AND (buyer_id = auth.uid() OR seller_id = auth.uid())
  )
);
```

Ratings RLS

```
CREATE POLICY "ratings_insert_buyer" ON public.ratings
FOR INSERT TO authenticated WITH CHECK (buyer_id = auth.uid());

CREATE POLICY "ratings_select_all" ON public.ratings
FOR SELECT USING (true);
```

Reports RLS

```
CREATE POLICY "reports_insert_auth" ON public.reports
FOR INSERT TO authenticated WITH CHECK (reporter_id = auth.uid());

CREATE POLICY "reports_select_own" ON public.reports
FOR SELECT TO authenticated USING (reporter_id = auth.uid());

CREATE POLICY "reports_delete_admin" ON public.reports
FOR DELETE USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND is_admin = true)
);
```

Notifications RLS

```
CREATE POLICY "notifications_select_own" ON public.notifications
FOR SELECT TO authenticated USING (user_id = auth.uid());

CREATE POLICY "notifications_update_own" ON public.notifications
FOR UPDATE TO authenticated USING (user_id = auth.uid());

CREATE POLICY "notifications_insert_all" ON public.notifications
FOR INSERT WITH CHECK (true);
```

Business Profiles RLS

```
CREATE POLICY "business_insert_all" ON public.business_profiles
FOR INSERT WITH CHECK (true);

CREATE POLICY "business_select_admin" ON public.business_profiles
FOR SELECT USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND is_admin = true)
);
```

Push Preferences RLS

```
CREATE POLICY "push_all_own" ON public.push_preferences
FOR ALL TO authenticated
USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
```

Triggers, Functions, and Cron Jobs

handle_new_user — creates profile row on registration

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
INSERT INTO public.profiles (
id, email, full_name, residence,
avatar_initials, avatar_color, joined_date
) VALUES (
NEW.id,
NEW.email,
COALESCE(NEW.raw_user_meta_data->>'full_name', ''),
NEW.raw_user_meta_data->>'residence',
COALESCE(NEW.raw_user_meta_data->>'avatar_initials', ''),
COALESCE(NEW.raw_user_meta_data->>'avatar_color', '#0F6E56'),
now()
) ON CONFLICT (id) DO NOTHING;
RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;
CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

update_seller_rating — recalculates avg_rating after each rating insert

```
CREATE OR REPLACE FUNCTION public.update_seller_rating()
RETURNS trigger AS $$
BEGIN
UPDATE public.profiles
SET
avg_rating = (SELECT ROUND(AVG(stars)::numeric, 1)
FROM public.ratings WHERE seller_id = NEW.seller_id),
total_ratings = (SELECT COUNT(*)
FROM public.ratings WHERE seller_id = NEW.seller_id)
WHERE id = NEW.seller_id;
RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS on_rating_inserted ON public.ratings;
CREATE TRIGGER on_rating_inserted
AFTER INSERT ON public.ratings
FOR EACH ROW EXECUTE FUNCTION public.update_seller_rating();
```

update_total_listings — keeps total_listings accurate on profile

```

CREATE OR REPLACE FUNCTION public.update_total_listings()
RETURNS trigger AS $$
BEGIN
UPDATE public.profiles
SET total_listings = (
SELECT COUNT(*) FROM public.listings
WHERE seller_id = COALESCE(NEW.seller_id, OLD.seller_id)
AND status IN ('active', 'pending')
)
WHERE id = COALESCE(NEW.seller_id, OLD.seller_id);
RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS on_listing_changed ON public.listings;
CREATE TRIGGER on_listing_changed
AFTER INSERT OR UPDATE OR DELETE ON public.listings
FOR EACH ROW EXECUTE FUNCTION public.update_total_listings();

```

increment_contact_count — RPC function called after conversation created

```

CREATE OR REPLACE FUNCTION increment_contact_count(listing_id UUID)
RETURNS void AS $$
UPDATE public.listings
SET contact_count = contact_count + 1
WHERE id = listing_id;
$$ LANGUAGE sql SECURITY DEFINER;

```

increment_report_count — RPC function called after report inserted

```

CREATE OR REPLACE FUNCTION increment_report_count(listing_id UUID)
RETURNS void AS $$
UPDATE public.listings
SET report_count = report_count + 1
WHERE id = listing_id;
$$ LANGUAGE sql SECURITY DEFINER;

```

Listing expiry cron job — runs every hour

```

SELECT cron.schedule(
'expire-listings',
'0 * * * *',
$$
UPDATE public.listings
SET status = 'expired'
WHERE expires_at < NOW()
AND status = 'active';
$$
);

```

Realtime publications — run after schema is created

```

ALTER PUBLICATION supabase_realtime ADD TABLE public.messages;
ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;

```

Storage bucket — listing-images

```
-- Create via Supabase Dashboard: Storage → New Bucket
-- Name: listing-images
-- Public: YES
-- Max file size: 5MB
-- Allowed MIME types: image/*,video/*

-- Or via SQL:
INSERT INTO storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
VALUES ('listing-images', 'listing-images', true, 5242880,
ARRAY['image/jpeg','image/png','image/webp','image/gif','video/mp4'])
ON CONFLICT (id) DO NOTHING;

CREATE POLICY "storage_insert_auth" ON storage.objects
FOR INSERT TO authenticated
WITH CHECK (bucket_id = 'listing-images');

CREATE POLICY "storage_select_public" ON storage.objects
FOR SELECT USING (bucket_id = 'listing-images');

CREATE POLICY "storage_delete_own" ON storage.objects
FOR DELETE TO authenticated
USING (bucket_id = 'listing-images' AND owner = auth.uid());
```

Supabase Auth settings — configure in Dashboard

```
-- Authentication → Providers → Email:
-- Enable Email: ON
-- Confirm Email: OFF (no email confirmation)
-- Secure Email Change: OFF

-- Authentication → Email Templates: not used, leave default

-- After first registration, grant yourself admin:
UPDATE public.profiles SET is_admin = true WHERE email = 'your-email@example.com';
```


6. Hard Rules for the Builder

No hardcoded data	Zero. If a list is empty, show an empty state message. Never fabricate listings, users, messages, or ratings. The only hardcoded values are plan tier constants in <code>dataService.ts</code> and the six avatar colours.
No duplicate function definitions	<code>resolveConversation()</code> and <code>markConversationResolved()</code> do the same thing. Use <code>markConversationResolved()</code> only. Delete <code>resolveConversation()</code> .
No direct Supabase calls in components	All database reads and writes go through <code>dataService.ts</code> functions. The only exception is the realtime channel subscription in <code>ChatPage</code> (messages) and <code>NotificationBell</code> (notifications), which use <code>supabase.channel()</code> directly because they need component lifecycle control.
No separate component files if avoidable	<code>ReportModal</code> lives in <code>ListingDetail.tsx</code> . <code>ChatWindow</code> lives in <code>ChatPage.tsx</code> . <code>RatingModal</code> lives in <code>ChatPage.tsx</code> . Only extract to a separate file if the component is used in more than one page.
No Navbar on admin pages	<code>AdminPanel</code> uses only <code>BottomNav</code> . No top Navbar.
No Navbar on student pages with BottomNav	Pages that render <code>BottomNav</code> do not also render Navbar on the same view — that creates double navigation. Navbar renders once in <code>App.tsx</code> layout. <code>BottomNav</code> renders in each page that needs it.
isLoadingAuth in every protected useEffect	Any <code>useEffect</code> that checks <code>currentUser</code> and redirects must include <code>isLoadingAuth</code> in its dependency array and return early if <code>isLoadingAuth</code> is true. Without this, the page shows a blank loading state permanently.
Empty states not loading spinners	After data fetch completes and returns empty, show a meaningful empty state message. Never leave the screen blank or spinning after fetch resolves.
Status values are fixed	The only valid listing status values are: pending, active, sold, expired, suspended. Never introduce rejected. suspended is what rejection sets.
Notification types are fixed	The only valid notification type values are: <code>listing_approved</code> , <code>listing_rejected</code> , <code>rating_request</code> . Do not invent new types.
public/ folder for static assets	<code>logo.png</code> must be in the <code>public/</code> folder at project root. Vite serves files from <code>public/</code> at the root URL path. A <code>logo.png</code> in <code>src/</code> or the project root will return 404.
Case-insensitive residence search	<code>getResidences()</code> returns names as stored. The <code>datalist/autocomplete</code> in <code>EditProfile</code> must match regardless of case. Use <code>toLowerCase()</code> on both sides when comparing for suggestions.